LAB MANUAL

Operating System

DEPARTMENT OF SOFTWARE ENGINEERING
FACULTY OF ENGINEERING & COMPUTING
NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD

TABLE OF CONTENTS

Preface

This lab manual has been prepared to facilitate the software engineering students in studying and implementing the concepts those underlying operating systems. Provide an opportunity to use basic concepts to solve real-world problems and the concepts learned through the implementation of the components of operating systems also explain common features of operating systems and explain the historical reasons why different features of operating systems were developed. Contrast batch, online (interactive) and real-time processing. Contrast real-time transaction processing and process-control operating systems. Differentiate between multiprocessing, multiprogramming, and multitasking.Explain the purpose and examples of spooling of input and output. Explain the developments of different versions of popular operating systems, including DOS/Windows and UNIX/Linux. Comparea monolithic kernel with a microkernel. Justify the use of layers of abstraction and explain the concept of hardware-OS boundary transparency. Explain the benefits of object-oriented design in operating systems.

Tools/ Technologies

Latest Version of Red hat Linux Enterprise Edition and Turbo C Compiler have been used for hands on experience by students in the labs

Lab No. 1: Introduction

Objectives

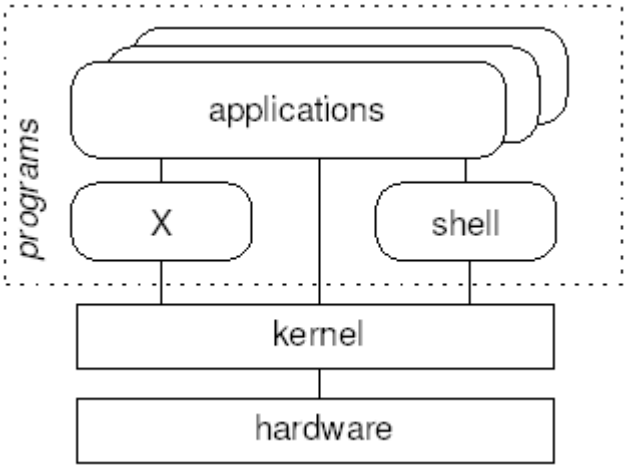
Objective of this lab is to give some background of Linux and elaborate the procedure of installing Linux.

Theoretical Description

Unix and Linux

- Linux is based on Unix
  - Unix philosophy
  - Unix commands
  - Unix standards and conventions
- There is some variation between Unix operating systems
  - Especially regarding system administration
  - Often Linux-specific things in these areas

Unix System Architecture



- The shell and the window environment are programs
- Programs' only access to hardware is via the kernel
- **Unix Philosophy**
- Multi-user

- A **user** needs an **account** to use a computer
- Each user must **log in**
- Complete separation of different users' files and configuration settings

- Small components
  - Each component should perform a single task
  - Multiple components can be combined and chained together for more complex tasks
  - An individual component can be substituted for another, without affecting other components

What is Linux?

- **Linux kernel**
  - Developed by Linus Torvalds
  - Strictly speaking, 'Linux' is just the kernel
- **Associated utilities**
  - Standard tools found on (nearly) all Linux systems
  - Many important parts come from the **GNU** project
    - Free Software Foundation's project to make a free Unix
    - Some claim the OS as a whole should be 'GNU/Linux'
- **Linux distributions**
    - Kernel plus utilities plus other tools, packaged up for end users
    - Generally, with installation program
  - Distributors include: Red Hat, Debian, SuSE, Mandrake

Installation

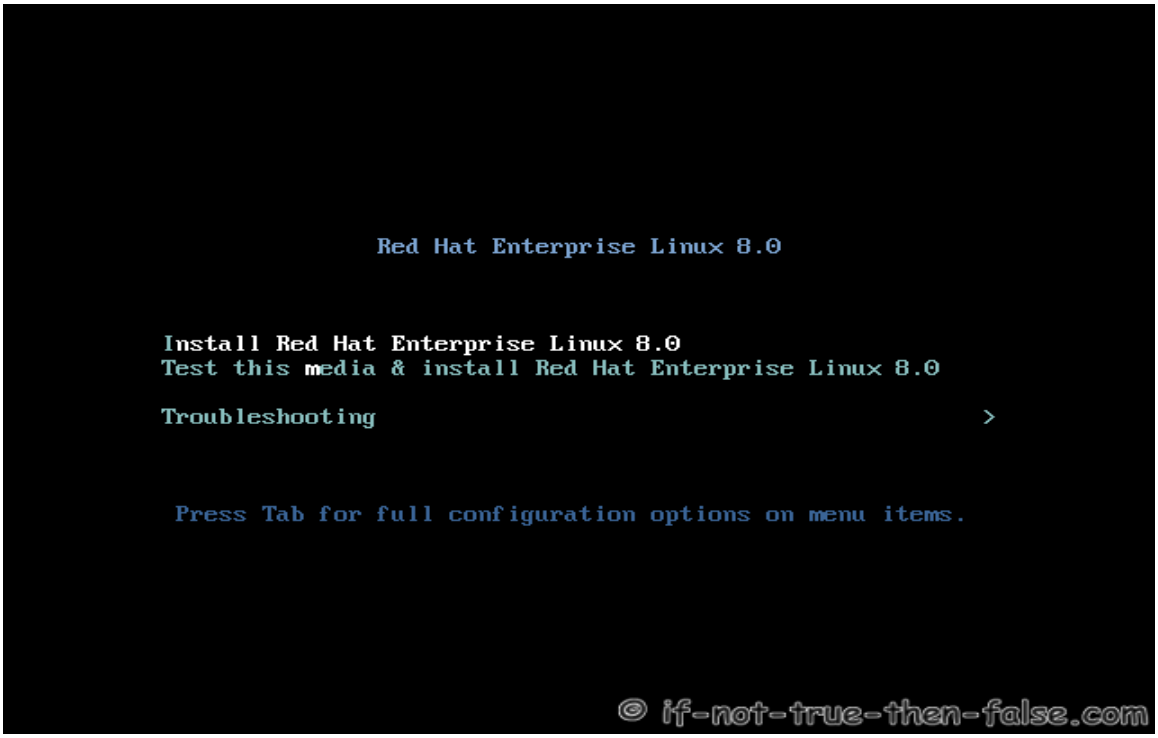
Pre installation instructions

- Free some space on your hard disk for installing Linux and delete it. Now when Linux installation will run you will have to select this unpartitioned area for your Linux installation...
- Perform media check on Linux installation CDs to confirm the integrity of the installing media.
- There are two modes of installation
  - Texture Interface for professionals

- Graphical User Interface for novice people
  - Its is recommended to select the latter option...

- At the time of partitioning, you will be prompted to select ‘manual partitioning’ or ‘automatic partitioning’. Automatic partitioning is recommended for new users.
- For running NS-2 or some other development tools later in Linux, it is recommended to install all development packages at the time of installation. Otherwise, you can install them later just like ‘add/remove window components’ in windows.
- Select the ‘Boot from CD option’ and start installation...
- To help you during the installation procedure, some tips are normally provided on the left top corner of the screen.

Following are the screen snapshots of Linux Redhat installation...



Lab Tasks

- What is the file system used by the Linux?
- Name two types of boot loaders available.
- What are the names of partitions created for Linux?

Lab No. 2: Basic commands of Linux

Objectives

This lab introduces few of the basic commands of Linux.

Theoretical Description

Getting Started with Linux

Using a Linux System

- Login prompt displayed
    - When Linux first loads after booting the computer
    - After another user has logged out
    - Need to enter a **username** and **password**
    - The login prompt may be graphical or simple text
    - If text, logging in will present a **shell**
    - If graphical, logging in will present a **desktop**
        - Some combination of mousing and keystrokes will make a **terminal window** appear
        - A shell runs in the terminal window

Linux Command Line

- The shell is where commands are invoked
- A command is typed at a **shell prompt**
    - Prompt usually ends in a dollar sign ($)
- After typing a command press Enter to invoke it
    - The shell will try to obey the command
    - Another prompt will appear
- Example:

    $ **date**

    Sat March 01 11:59:05 BST 2008

    $

    - The dollar represents the prompt in this course, do not type it

Logging Out

- To exit from the shell, use the exit command
- Pressing Ctrl+D at the shell prompt will also quit the shell
  - Quitting all programs should log you out
  - If in a text-only single-shell environment, exiting the shell should be sufficient
- In a window environment, the window manager should have a log out command for this purpose
- After logging out, a new login prompt should be displayed

Command Syntax

- Most commands take **parameters**
  - Some commands *require* them
  - Parameters are also known as **arguments**
  - For example, echo simply displays its arguments:

    $ **echo**

    $ **echo Hello there**

    Hello there
- Commands are case-sensitive
  - Usually lower-case

    $ **echo whisper**

    whisper

    $ **ECHO SHOUT**

    bash: ECHO: command not found

Files

- Data can be stored in a **file**
- Each file has a **filename**
  - A label referring to a particular file
  - Permitted characters include letters, digits, hyphens (-), underscores (_), and dots (.)
  - Case-sensitive — *NewsCrew.mov* is a different file from *NewScrew.mov*
- The ls command lists the names of files

Creating Files with cat

- There are many ways of creating a file
- One of the simplest is with the cat command:

  $ **cat > shopping _list**

  cucumber

  bread

  yoghurts

  fish fingers
- Note the greater-than sign (>) — this is necessary to create the file
- The text typed is written to a file with the specified name
- Press Ctrl+D after a line-break to denote the end of the file
  - The next shell prompt is displayed
- ls demonstrates the existence of the new file

Displaying Files' Contents with cat

- There are many ways of viewing the contents of a file
- One of the simplest is with the cat command:

  $ **cat shopping_ list**

  cucumber

  bread

  yoghurts

  fish fingers
- Note that no greater-than sign is used
- The text in the file is displayed immediately:
  - Starting on the line after the command
  - Before the next shell prompt

Deleting Files with rm

- To delete a file, use the rm ('remove') command
- Simply pass the name of the file to be deleted as an argument:

  $ **rm shopping_ list**
- The file and its contents are removed
  - There is no recycle bin
  - There is no 'unrm' command

- The ls command can be used to confirm the deletion

Unix Command Feedback

- Typically, successful commands do not give any output
- Messages are displayed in the case of errors
- The rm command is typical
    - If it manages to delete the specified file, it does so silently
    - There is no 'File shopping list has been removed' message
    - But if the command fails for whatever reason, a message is displayed
- The silence can be off-putting for beginners
- It is standard behavior, and doesn't take long to get used to

Copying and Renaming Files with cp and mv

- To copy the contents of a file into another file, use the cp command:

  $ **cp CV.pdf old-CV.pdf**
- To rename a file use the mv ('move') command:

  $ **mv commitee_minutes.txt committee_minutes.txt**
    - Similar to using cp then rm
- For both commands, the existing name is specified as the first argument and the new name as the second
    - If a file with the new name already exists, it is overwritten

Filename Completion

- The shell can make typing filenames easier
- Once an unambiguous prefix has been typed, pressing Tab will automatically 'type' the rest
- For example, after typing this:

  $ **rm sho**

  pressing Tab may turn it into this:

  $ **rm shopping_list**
- This also works with command names
    - For example, da may be completed to date if no other commands start 'da'

Command History

- Often it is desired to repeat a previously-executed command
- The shell keeps a **command history** for this purpose

- Use the *Up* and Down cursor keys to scroll through the list of previous commands
- Press *Enter* to execute the displayed command

- Commands can also be edited before being run
  - Particularly useful for fixing a typo in the previous command
  - The Left and Right cursor keys navigate across a command
  - Extra characters can be typed at any point
  - Backspace deletes characters to the left of the cursor
  - Del and Ctrl+D delete characters to the right
    - Take care not to log out by holding down Ctrl+D too long

Skills Developed

By completing the second lab, one should have basic understanding of Linux environment and few Linux commands.

Lab Tasks

TASK-1

- Log in.
- Log out.
- Log in again. Open a terminal window, to start a shell.
- Exit from the shell; the terminal window will close.
- Start another shell. Enter each of the following commands in turn.
  i. date
  ii. whoami
  iii. hostname
  iv. uname
  v. uptime

TASK-2

- Use the ls command to see if you have any files.
- Create a new file using the cat command as follows:

  $ **cat > hello.txt**

  Hello world!

  This is a text file.
- Press Enter at the end of the last line, then Ctrl+D to denote the end of the file.

- Use ls again to verify that the new file exists.
- Display the contents of the file.
- Display the file again, but use the cursor keys to execute the same command again without having to retype it.

TASK-3

- Create a second file. Call it *secret-of-the-universe*, and put in whatever content you deem appropriate.
- Check its creation with ls.
- Display the contents of this file. Minimize the typing needed to do this:
    i. Scroll back through the command history to the command you used to create the file.
    ii. Change that command to display *secret-of-the-universe* instead of creating it.

TASK-4

After each of the following steps, use ls and cat to verify what has happened.

- Copy *secret-of-the-universe* to a new file called *answer.txt*. Use Tab to avoid typing the existing file's name in full.
- Now copy *hello.txt* to *answer.txt*. What's happened now?
- Delete the original file, *hello.txt*.
- Rename *answer.txt* to *message*.
- Try asking rm to delete a file called *missing*. What happens?
- Try copying *secret-of-the-universe* again, but don't specify a filename to which to copy. What happens now?

Lab No. 3: Linux file system

Objectives

In this lab, you will explore the Linux file system, including the basic concepts of files and directories and their organization in a hierarchical tree structure.

Theoretical Description

File and Directories

- A **directory** is a collection of files and/or other directories
  - Because a directory can contain other directories, we get a directory **hierarchy**
- The 'top level' of the hierarchy is the **root directory**
- Files and directories can be named by a **path**
  - Shows programs how to find their way to the file
  - The root directory is referred to as /
  - Other directories are referred to by name, and their names are separated by slashes (/)
- If a path refers to a directory it can end in /
  - Usually, an extra slash at the end of a path makes no difference

Linux Files and Directories

Examples of Absolute Paths

- An **absolute path** starts at the root of the directory hierarchy, and names directories under it:
  /etc/hostname
  - Meaning the file called *hostname* in the directory *etc* in the root directory
- We can use ls to list files in a specific directory by specifying the absolute path:
  $ **ls /usr/share/doc/**

Current Directory

- Your shell has a **current directory** — the directory in which you are currently working

- Commands like ls use the current directory if none is specified
- Use the pwd (print working directory) command to see what your current directory is:

  $ **pwd**

  /home/fred
- Change the current directory with cd:

  $ **cd /mnt/cdrom**

  $ **pwd**

  /mnt/cdrom
- Use cd without specifying a path to get back to your home directory

Making and Deleting Directories

- The mkdir command makes new, empty, directories
- For example, to make a directory for storing company accounts:

  $ **mkdir Accounts**
- To delete an empty directory, use rmdir:

  $ **rmdir OldAccounts**
- Use rm with the -r (recursive) option to delete directories and all the files they contain:

  $ **rm -r OldAccounts**
- Be careful — rm can be a dangerous tool if misused

Relative Paths

- Paths don't have to start from the root directory
  - A path which doesn't start with / is a **relative path**
  - It is relative to some other directory, usually the current directory
- For example, the following sets of directory changes both end up in the same directory:

  $ **cd /usr/share/doc**

  $ **cd /**

  $ **cd usr**

  $ **cd share/doc**
- Relative paths specify files inside directories in the same way as absolute ones

Special Dot Directories

- Every directory contains two special filenames which help making relative paths:
  - The directory .. points to the parent directory
    - ls .. will list the files in the parent directory
  - For example, if we start from */home/fred*:

    $ **cd ..**

    $ **pwd**

    /home

    $ **cd ..**

    $ **pwd**

    /
- The special directory . points to the directory it is in
  - So ./foo is the same file as foo

Using Dot Directories in Paths

- The special .. and . directories can be used in paths just like any other directory name:

  $ **cd ../other-dir/**
  - Meaning "the directory *other-dir* in the parent directory of the current directory"
- It is common to see .. used to 'go back' several directories from the current directory:

  $ **ls ../../../../far-away-directory/**
- The . directory is most commonly used on its own, to mean "the current directory"

Hidden Files

- The special . and .. directories don't show up when you do ls
  - They are **hidden files**
- Simple rule: files whose names start with . are considered 'hidden'
- Make ls display all files, even the hidden ones, by giving it the -a (all) option:

  $ **ls -a**

  . .. .bashrc .profile report.doc

- Hidden files are often used for configuration files
  - Usually found in a user's home directory
- You can still read hidden files — they just don't get listed by ls by default

Paths to Home Directories

- The symbol ~ (tilde) is an abbreviation for your home directory
  - So for user 'fred', the following are equivalent:

  $ **cd /home/fred/documents/**

  $ **cd ~/documents/**
- The ~ is **expanded** by the shell, so programs only see the complete path
- You can get the paths to other users' home directories using ~, for example:

  $ **cat ~alice/notes.txt**
- The following are all the same for user 'fred':

  $ **cd**

  $ **cd ~**

  $ **cd /home/fred**

Looking for Files in the System

- The command locate lists files which contain the text you give
- For example, to find files whose name contains the word 'mkdir':

  $ **locate mkdir**

  /usr/man/man1/mkdir.1.gz

  /usr/man/man2/mkdir.2.gz

  /bin/mkdir

  ...
- locate is useful for finding files when you don't know exactly what they will be called, or where
- they are stored
- For many users, graphical tools make it easier to navigate the filesystem
  - Also make file management simpler

Running Programs

- Programs under Linux are files, stored in directories like */bin* and */usr/bin*
  - Run them from the shell, simply by typing their name
- Many programs take options, which are added after their name and prefixed with -

- For example, the -l option to ls gives more information, including the size of files and the date
- they were last modified:

  $ **ls -l**

drwxrwxr-x	2	fred	users	4096	Mar 01	10:57	Accounts
-rw-rw-r--	1	fred	users	345	Mar 01	10:57	notes.txt
-rw-r--r--	1	fred	users	3255	Mar 01	10:57	report.txt

- Many programs accept filenames after the options
  - Specify multiple files by separating them with spaces

Specifying Multiple Files

- Most programs can be given a list of files
  - For example, to delete several files at once:

    $ **rm oldnotes.txt tmp.txt stuff.doc**
  - To make several directories in one go:

    $ **mkdir Accounts Reports**
- The original use of cat was to join multiple files together
  - For example, to list two files, one after another:

    $ **cat notes.txt morenotes.txt**
- If a filename contains spaces, or characters which are interpreted by the shell (such as *), put
- single quotes around them:

  $ **rm 'Beatles - Strawberry Fields.mp3'**

  $ **cat '* important notes.txt *'**

Finding Documentation for Programs

- Use the man command to read the manual for a program
- The manual for a program is called its **man page**
  - Other things, like file formats and library functions also have man pages
- To read a man page, specify the name of the program to man:

  $ **man mkdir**
- To quit from the man page viewer press q
- Man pages for programs usually have the following information:
  - A description of what it does

- A list of options it accepts
- Other information, such as the name of the author

Specifying Files with Wildcards

- Use the * wildcard to specify multiple filenames to a program:

  $ **ls -l *.txt**

  -rw-rw-r-- 1 fred users 108 Nov 16 13:06 report.txt

  -rw-rw-r-- 1 fred users 345 Jan 18 08:56 notes.txt
- The shell expands the wildcard, and passes the full list of files to the program
- Just using * on its own will expand to all the files in the current directory:

  $ **rm ***
  - (All the files, that is, except the hidden ones)
- Names with wildcards in are called **globs**, and the process of expanding them is called **globbing**

Chaining Programs Together

- The who command lists the users currently logged in
- The wc command counts bytes, words, and lines in its input
- We combine them to count how many users are logged in:

  $ **who | wc -l**
- The | symbol makes a **pipe** between the two programs
  - The output of who is fed into wc
- The -l option makes wc print only the number of lines
- Another example, to join all the text files together and count the words, lines and characters in
- the result:

  $ **cat *.txt | wc**

Graphical and Text Interfaces

- Most modern desktop Linux systems provide a **graphical user interface** (GUI)
- Linux systems use the X window system to provide graphics
  - X is just another program, not built into Linux
  - Usually, X is started automatically when the computer boots
- Linux can be used without a GUI, just using a command line
- Use Ctrl+Alt+F1 to switch to a text console — logging in works as it does in X

- Use Ctrl+Alt+F2, Ctrl+Alt+F3, etc., to switch between virtual terminals — usually about 6 are provided
- Use Ctrl+Alt+F7, or whatever is after the virtual terminals, to switch back to X

Text Editors

- Text editors are for editing plain text files
  - Don't provide advanced formatting like word processors
  - Extremely important — manipulating text is Unix's *raison d'être*
- The most popular editors are Emacs and Vim, both of which are very sophisticated, but take
- time to learn
- Simpler editors include Nano, Pico, Kedit and Gnotepad
- Some programs run a text editor for you
  - They use the $EDITOR variable to decide which editor to use
  - Usually it is set to vi, but it can be changed
  - Another example of the component philosophy

Lab Tasks

Task-1

I. Use the pwd command to find out what directory you are in.

II. If you are not in your home directory (*/home/USERNAME*) then use cd without any arguments to go there, and do pwd again.

III. Use cd to visit the root directory, and list the files there. You should see *home* among the list.

IV. Change into the directory called *home* and again list the files present. There should be one directory for each user, including the user you are logged in as (you can use whoami to check that).

V. Change into your home directory to confirm that you have gotten back to where you started.

Task-2

I. Create a text file in your home directory called *shakespeare*, containing the following text:

Shall I compare thee to a summer's day?

Thou art more lovely and more temperate

II. Rename it to *sonnet-18.txt*.

III. Make a new directory in your home directory, called *poetry*.

IV. Move the poem file into the new directory.

V. Try to find a graphical directory-browsing program, and find your home directory with it. You should also be able to use it to explore some of the system directories.

VI. Find a text editor program and use it to display and edit the sonnet.

Task-3

I. From your home directory, list the files in the directory */usr/share*.

II. Change to that directory, and use pwd to check that you are in the right place. List the files in the current directory again, and then list the files in the directory called *doc*.

III. Next list the files in the parent directory, and the directory above that.

IV. Try the following command, and make sure you understand the result:

a. $ echo ~

V. Use cat to display the contents of a text file which resides in your home directory (create one if you haven't already), using the ~/ syntax to refer to it. It shouldn't matter what your current directory is when you run the command.

Task-4

I. Use the hostname command, with no options, to print the hostname of the machine you are using.

II. Use man to display some documentation on the hostname command. Find out how to make it print the IP address of the machine instead of the hostname. You will need to scroll down the manpage to the 'Options' section.

III. Use the locate command to find files whose name contains the text 'hostname'. Which of the filenames printed contain the actual hostname program itself? Try running it by entering the program's absolute path to check that you really have found it.

Task-5

I. The * wildcard on its own is expanded by the shell to a list of all the files in the current directory. Use the echo command to see the result (but make sure you are in a directory with a few files or directories first)

II. Use quoting to make echo print out an actual * symbol.

III. Augment the *poetry* directory you created earlier with another file, *sonnet-29.txt*:

   a. When in disgrace with Fortune and men's eyes,

   b. I all alone beweep my outcast state,

IV. Use the cat command to display both of the poems, using a wildcard.

V. Finally, use the rm command to delete the *poetry* directory and the poems in it.

Lab No. 4: Overview of Linux Shell

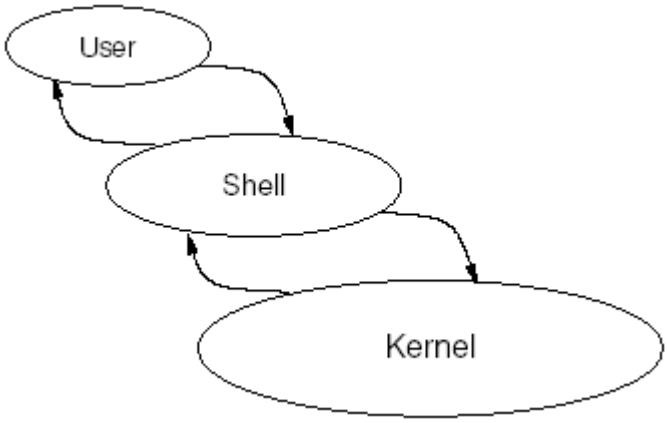
Objectives

This lab will give overview of Linux shells. You will get insight of 'bash' shell.

Theoretical Description

Shells

- A **shell** provides an interface between the user and the operating system kernel
- Either a **command interpreter** or a graphical user interface
- Traditional Unix shells are **command-line interfaces** (CLIs)
- Usually started automatically when you log in or open a terminal



Work Effectively on the Linux Command Line

The Bash Shell

Linux's most popular command interpreter is called bash

- The **Bourne-Again Shell**
  - More sophisticated than the original sh by Steve Bourne
  - Can be run as sh, as a replacement for the original Unix shell
  - Gives you a prompt and waits for a command to be entered
- Although this course concentrates on Bash, the shell tcsh is also popular
  - Based on the design of the older C Shell (csh)

Shell Commands

- Shell commands entered consist of words
  - Separated by spaces (whitespace)
  - The first word is the command to run
  - Subsequent words are options or arguments to the command
- For several reasons, some commands are built into the shell itself
  - Called **builtins**
  - Only a small number of commands are builtins, most are separate programs

Command-Line Arguments

- The words after the command name are passed to a command as a list of **arguments**
- Most commands group these words into two categories:
  - Options, usually starting with one or two hyphens
  - Filenames, directories, etc., on which to operate

- The options usually come first, but for most commands they do not need to
- There is a special option '--' which indicates the end of the options
  - Nothing after the double hyphen is treated as an option, even if it starts with -

Syntax of Command-Line Options

- Most Unix commands have a consistent syntax for options:
  - Single letter options start with a hyphen, e.g., -B
  - Less cryptic options are whole words or phrases, and start with two hyphens, for example

    --ignore-backups
- Some options themselves take arguments
  - Usually the argument is the next word: sort -o *output_file*
- A few programs use different styles of command-line options
  - For example, long options (not single letters) sometimes start with a single – rather than --

Examples of Command-Line Options

- List all the files in the current directory:

  $ **ls**

- List the files in the 'long format' (giving more information):

  $ **ls -l**

- List full information about some specific files:

  $ **ls -l notes.txt report.txt**

- List full information about all the *.txt* files:

  $ **ls -l *.txt**

- List all files in long format, even the hidden ones:

  $ **ls -l -a**

  $ **ls -la**

Setting Shell Variables

- **Shell variables** can be used to store temporary values
- Set a shell variable's value as follows:

  $ **files="notes.txt report.txt"**

  - The double quotes are needed because the value contains a space
  - Easiest to put them in all the time
- Print out the value of a shell variable with the echo command:

  $ **echo $files**

  - The dollar ($) tells the shell to insert the variable's value into the command line
- Use the set command (with no arguments) to list all the shell variables

Environment Variables

- Shell variables are private to the shell
- A special type of shell variables called **environment variables** are passed to programs run from the shell
- A program's **environment** is the set of environment variables it can access
  - In Bash, use export to export a shell variable into the environment:

    $ **files="notes.txt report.txt"**

    $ **export files**

- Or combine those into one line:

  $ **export files="notes.txt report.txt"**

- The env command lists environment variables

Where Programs are Found

- The location of a program can be specified explicitly:
  - ./sample runs the sample program in the current directory
  - /bin/ls runs the ls command in the */bin* directory
- Otherwise, the shell looks in standard places for the program
  - The variable called PATH lists the directories to search in
  - Directory names are separated by colon, for example:

    $ **echo $PATH**

    /bin:/usr/bin:/usr/local/bin
  - So running whoami will run */bin/whoami* or */usr/bin/whoami* or */usr/local/bin/whoami* (whichever is found first)

Bash Configuration Variables

- Some variables contain information which Bash itself uses
  - The variable called PS1 (Prompt String 1) specifies how to display the shell prompt
- Use the echo command with a $ sign before a varable name to see its value, e.g.

  $ **echo $PS1**

  [\u@\h \W]\$
- The special characters \u, \h and \W represent shell variables containing, respectively, your user/login name, machine's hostname and current working directory, i.e.,
  - $USER, $HOSTNAME, $PWD

Using History

- Previously executed commands can be edited with the Up or Ctrl+P keys
- This allows old commands to be executed again without re-entering
- Bash stores a **history** of old commands in memory
  - Use the built-in command history to display the lines remembered
  - History is stored between sessions in the file *~/.bash_history*
- Bash uses the readline library to read input from the user

- Allows Emacs-like editing of the command line
- Left and Right cursor keys and Delete work as expected

Reusing History Items

- Previous commands can be used to build new commands, using **history expansion**
- Use !! to refer to the previous command, for example:

  $ **rm index.html**

  $ **echo !!**

  echo rm index.html

  rm index.html
- More often useful is !*string*, which inserts the most recent command which started with *string*
  - Useful for repeating particular commands without modification:

    $ **ls *.txt**

    notes.txt report.txt

    $ **!ls**

    ls *.txt

    notes.txt report.txt

Retrieving Arguments from the History

- The event designator !$ refers to the last argument of the previous command:

  $ **ls -l long_file_name.html**

  -rw-r--r-- 1 jeff users 11170 Feb 20 10:47 long_file_name.html

  $ **rm !$**

  rm long_file_name.html
- Similarly, !^ refers to the first argument
- A command of the form ^*string*^*replacement*^ replaces the first occurrence of *string* with *replacement* in the previous command, and runs it:

  $ **echo $HOTSNAME**

  $ **^TS^ST^**

  echo $HOSTNAME

  tiger

Summary of Bash Editing Keys

- These are the basic editing commands by default:
  - Right — move cursor to the right
  - Left — move cursor to the left
  - Up — previous history line
  - Down — next history line
  - Ctrl+A — move to start of line
  - Ctrl+E — move to end of line
  - Ctrl+D — delete current character
- There are alternative keys, as for the Emacs editor, which can be more comfortable to use than the cursor keys
- There are other, less often used keys, which are documented in the bash man page (section 'Readline')

Combining Commands on One Line

- You can write multiple commands on one line by separating them with ;
- Useful when the first command might take a long time:
  time-consuming-program; ls
- Alternatively, use && to arrange for subsequent commands to run only if earlier ones succeeded:
  time-consuming-potentially-failing-program && ls

Repeating Commands with 'for'

- Commands can be repeated several times using for
  - Structure: for *varname* in *list*; do *commands...*; done
- For example, to rename all *.txt* files to *.txt.old* :
  $ **for file in *.txt;**
  > do
  > mv -v $file $file.old;
  > done
  barbie.txt -> barbie.txt.old
  food.txt -> food.txt.old
  quirks.txt -> quirks.txt.old
- The command above could also be written on a single line

Command Substitution

- **Command substitution** allows the output of one command to be used as arguments to another
- For example, use the locate command to find all files called *manual.html* and print information about them with ls:

  $ **ls -l $(locate manual.html)**

  $ **ls -l 'locate manual.html'**
- The punctuation marks on the second form are opening single quote characters, called **backticks**
  - The $() form is usually preferred, but backticks are widely used
- Line breaks in the output are converted to spaces
- Another example: use vi to edit the last of the files found:

  $ **vi $(locate manual.html | tail -1)**

Finding Files with locate

- The locate command is a simple and fast way to find files
- For example, to find files relating to the email program mutt:

  $ **locate mutt**
- The locate command searches a database of filenames
  - The database needs to be updated regularly
  - Usually this is done automatically with cron
  - But locate will not find files created since the last update
- The -i option makes the search case-insensitive
- -r treats the pattern as a regular expression, rather than a simple string

Finding Files More Flexibly: 'find'

- locate only finds files by name
- *find* can find files by any combination of a wide number of criteria, including name
- Structure: *find directories criteria*
- Simplest possible example: find .
- Finding files with a simple criterion:

  $ **find . -name manual.html**

  Looks for files under the current directory whose name is *manual.html*

- The *criteria* always begin with a single hyphen, even though they have long names

'find' Criteria

- find accepts many different criteria; two of the most useful are:
  - -name *pattern*: selects files whose name matches the shell-style wildcard *pattern*
  - -type d, -type f: select directories or plain files, respectively
- You can have complex selections involving 'and', 'or', and 'not'

'find' Actions: Executing Programs

- *find* lets you specify an action for each file found; the default action is simply to print out the name
  - You can alternatively write that explicitly as -print
- Other actions include executing a program; for example, to delete all files whose name starts
- with *manual*:

  find . -name 'manual*' -exec rm '{}' ';'
- The command rm '{}' is run for each file, with '{}' replaced by the filename
- The {} and ; are required by *find*, but must be quoted to protect them from the shell

Lab Tasks

Task-1

I. Use the df command to display the amount of used and available space on your hard drive.

II. Check the man page for df, and use it to find an option to the command which will display the free space in a more human-friendly form. Try both the single-letter and long-style options.

III. Run the shell, bash, and see what happens. Remember that you were already running it to start with. Try leaving the shell you have started with the exit command.

Task-2

I. Try ls with the -a and -A options. What is the difference between them?

II. Write a for loop which goes through all the files in a directory and prints out their names with echo. If you write the whole thing on one line, then it will be easy to repeat it using the command line history.

III. Change the loop so that it goes through the names of the people in the room (which needn't be the names of files) and print greetings to them.

IV. Of course, a simpler way to print a list of filenames is echo *. Why might this be useful, when we usually use the ls command?

Task-3

I. Use the find command to list all the files and directories under your home directory. Try the -type d and -type f criteria to show just files and just directories.

II. Use 'locate' to find files whose name contains the string 'bashbug'. Try the same search with find, looking over all files on the system. You'll need to use the * wildcard at the end of the pattern to match files with extensions.

III. Find out what the find criterion -iname does.

Lab No. 5: Basic File and Directory Management Utilities

Objectives

This lab will give overview of Basic File and Directory Management utilities.

Theoretical Description

Filesystem Objects

- A **file** is a place to store data: a possibly-empty sequence of bytes
- A **directory** is a collection of files and other directories
- Directories are organized in a hierarchy, with the **root directory** at the top
- The root directory is referred to as /

home/

/

bin/

cp rm jeff/

Directory and File Names

- Files and directories are organized into a **filesystem**
- Refer to files in directories and sub-directories by separating their names with /, for example:

/bin/ls

/usr/share/dict/words

/home/jeff/recipe

- Paths to files either start at / (absolute) or from some 'current' directory

File Extensions

- It's common to put an **extension**, beginning with a dot, on the end of a filename
- The extension can indicate the type of the file:

.txt Text file

.gif Graphics Interchange Format image

.jpg Joint Photographic Experts Group image

.mp3 MPEG-2 Layer 3 audio

.gz Compressed file

.tar Unix 'tape archive' file

.tar.gz, *.tgz* Compressed archive file

- On Unix and Linux, file extensions are just a convention
- The kernel just treats them as a normal part of the name
- A few programs use extensions to determine the type of a file

Going Back to Previous Directories

- The pushd command takes you to another directory, like cd
- But also saves the current directory, so that you can go back later
- For example, to visit Fred's home directory, and then go back to where you started from:

$ pushd ~fred

$ cd Work

$ ls

...

$ popd

- popd takes you back to the directory where you last did pushd
- dirs will list the directories you can pop back to

Filename Completion

- Modern shells help you type the names of files and directories by completing partial names
- Type the start of the name (enough to make it unambiguous) and press Tab
- For an ambiguous name (there are several possible completions), the shell can list the options:
- For Bash, type Tab twice in succession
- For C shells, type Ctrl+D
- Both of these shells will automatically escape spaces and special characters in the filenames

Wildcard Patterns

- Give commands multiple files by specifying patterns
- Use the symbol * to match any part of a filename:

  $ ls *.txt

  accounts.txt letter.txt report.txt

- Just * produces the names of all files in the current directory
- The wildcard ? matches exactly one character:

  $ rm -v data.?

  removing data.1

  removing data.2

  removing data.3
- Note: wildcards are turned into filenames by the shell, so the program you pass them to can't

tell that those names came from wildcard expansion

Copying Files with cp

- Syntax: cp [*options*] *source-file destination-file*
- Copy multiple files into a directory: cp *files directory*
- Common options:
- -f, force overwriting of destination files
- -i, interactively prompt before overwriting files
- -a, archive, copy the contents of directories recursively

Examples of cp

- Copy */etc/smb.conf* to the current directory:

$ cp /etc/smb.conf .

- Create an identical copy of a directory called *work*, and call it *work-backup*:

  $ cp -a work work-backup
- Copy all the GIF and JPEG images in the current directory into *images*:

$ cp *.gif *.jpeg images/

Moving Files with mv

- mv can rename files or directories, or move them to different directories
- It is equivalent to copying and then deleting
- But is usually much faster
- Options:
- -f, force overwrite, even if target already exists
- -i, ask user interactively before overwriting files
- For example, to rename *poetry.txt* to *poems.txt*:

  $ mv poetry.txt poems.txt

- To move everything in the current directory somewhere else:

  $ mv * ~/old-stuff/

Deleting Files with rm

- rm deletes ('removes') the specified files
- You must have to write permission for the directory the file is in to remove it
- Use carefully if you are logged in as root!
- Options:
- -f, delete write-protected files without prompting
- -i, interactive — ask the user before deleting files
- -r, recursively delete files and directories
- For example, clean out everything in */tmp*, without prompting to delete each file:

  $ rm -rf /tmp/*

Deleting Files with Peculiar Names

- Some files have names which make them hard to delete
- Files that begin with a minus sign:

  $ rm ./-filename

  $ rm -- -filename
- n Files that contain peculiar characters — perhaps characters that you can't actually type on your keyboard:
  - Write a wildcard pattern that matches *only* the name you want to delete:

    $ rm -i ./name-with-funny-characters*
  - The ./ forces it to be in the current directory
  - Using the -i option to rm makes sure that you won't delete anything else by accident

Making Directories with mkdir

- Syntax: mkdir *directory-names*
- Options:
  - -p, create intervening parent directories if they don't already exist
  - -m *mode*, set the access permissions to *mode*
- For example, create a directory called *mystuff* in your home directory with permissions so that only you can write, but eveyone can read it:

$ mkdir -m 755 ~/mystuff

- Create a directory tree in */tmp* using one command with three subdirectories called *one*, *two* and *three*:

$ mkdir -p /tmp/one/two/three

Removing Directories with rmdir

- rmdir deletes *empty* directories, so the files inside must be deleted first
- For example, to delete the *images* directory:

$ rm images/*

$ rmdir images

- For non-empty directories, use rm -r *directory*
- The -p option to rmdir removes the complete path, if there are no other files and directories in it
  - These commands are equivalent:

    $ rmdir -p a/b/c

    $ rmdir a/b/c a/b a

Identifying Types of Files

- The data in files comes in various different formats (executable programs, text files, etc.)
- nThe file command will try to identify the type of a file:

$ file /bin/bash

/bin/bash: ELF 32-bit LSB executable, Intel 80386, version 1,

dynamically linked (uses shared libs), stripped

- It also provides extra information about some types of files
- Useful to find out whether a program is actually a script:

  $ file /usr/bin/zless

  /usr/bin/zless: Bourne shell script text
- If file doesn't know about a specific format, it will guess:

  $ file /etc/passwd

  /etc/passwd: ASCII text

Changing Timestamps with touch

- Changes the **access** and **modification** times of files
- Creates files that didn't already exist

- Options:
  - -a, change only the access time
  - -m, change only the modification time
  - -t [*YYYY*]*MMDDhhmm*[*.ss*], set the timestamp of the file to the specified date and time
  - GNU touch has a -d option, which accepts times in a more flexible format
- For example, change the time stamp on *homework* to January 20 2001, 5:59p.m.

  $ touch -t 200101201759 homework

Lab Tasks

Task-1

I. Use cd to go to your home directory, and create a new directory there called *dog*.

II. Create another directory within that one called *cat*, and another within that called *mouse*.

III. Remove all three directories. You can either remove them one at a time, or all at once.

IV. If you can delete directories with rm -r, what is the point of using rmdir for empty directories?

V. Try creating the *dog/cat/mouse* directory structure with a single command.

Task-2

I. Copy the file */etc/passwd* to your home directory, and then use cat to see what's in it.

II. Rename it to *users* using the mv command.

III. Make a directory called *programs* and copy everything from */bin* into it.

IV. Delete all the files in the *program's* directory.

V. Delete the empty *programs* directory and the *user's* file.

Task-3

I. The touch command can be used to create new empty files. Try that now, picking a name for the new file:

$ touch baked-beans

II. Get details about the file using the ls command:

$ ls -l baked-beans

III. Wait for a minute, and then try the previous two steps again, and see what changes. What happens when we don't specify a time to touch?

IV. Try setting the timestamp on the file to a value in the future.
When you're finished with it, delete the file.

Lab No. 6: Process Text Streams

Objectives

This lab will give overview of Processing Text Streams using Text Processing Filters

Theoretical Description

Working with Text Files

- Unix-like systems are designed to manipulate text very well
- The same techniques can be used with plain text, or text-based formats
  - Most Unix configuration files are plain text
- Text is usually in the **ASCII** character set
  - Non-English text might use the ISO-8859 character sets
  - Unicode is better, but unfortunately many Linux command-line utilities don't (directly)

support it yet

Lines of Text

- Text files are naturally divided into lines
- In Linux a line ends in a **line feed** character
  - Character number 10, hexadecimal 0x0A
- Other operating systems use different combinations
  - Windows and DOS use a carriage return followed by a line feed
  - Macintosh systems use only a carriage return
  - Programs are available to convert between the various formats

Filtering Text and Piping

- The Unix philosophy: use small programs, and link them together as needed
- Each tool should be good at one specific job
- Join programs together with **pipes**
  - Indicated with the pipe character: |
  - The first program prints text to its **standard output**
  - That gets fed into the second program's **standard input**
- For example, to connect the output of echo to the input of wc:

  $ **echo "count these words, boy" | wc**

Displaying Files with less

- n If a file is too long to fit in the terminal, display it with less:

$ **less README**

- less also makes it easy to clear the terminal of other things, so is useful even for small files
- Often used on the end of a pipe line, especially when it is not known how long the output will be:

  $ **wc *.txt | less**
- Doesn't choke on strange characters, so it won't mess up your terminal (unlike cat)

Counting Words and Lines with wc

- wc counts characters, words and lines in a file
- If used with multiple files, outputs count for each file, and a combined total
- Options:
  - -c output character count
  - -l output line count
  - -w output word count
  - Default is -clw
- Examples: display word count for *essay.txt*:

  $ **wc -w essay.txt**
- Display the total number of lines in several text files:

  $ **wc -l *.txt**

Sorting Lines of Text with sort

- The sort filter reads lines of text and prints them sorted into order
- For example, to sort a list of words into dictionary order:

  $ **sort words > sorted-words**
- The -f option makes the sorting **case-insensitive**
- The -n option sorts numerically, rather than lexicographically

Removing Duplicate Lines with uniq

- Use uniq to find unique lines in a file
  - Removes *consecutive* duplicate lines
  - Usually give it sorted input, to remove all duplicates

- Example: find out how many unique words are in a dictionary:

  $ **sort /usr/dict/words | uniq | wc -w**

- sort has a -u option to do this, without using a separate program:

  $ **sort -u /usr/dict/words | wc -w**

- sort | uniq can do more than sort -u, though:
  - uniq -c counts how many times each line appeared
  - uniq -u prints only unique lines
  - uniq -d prints only duplicated lines

Selecting Parts of Lines with cut

- Used to select columns or fields from each line of input
- Select a range of
  - Characters, with -c
  - Fields, with -f
- Field separator specified with -d (defaults to tab)
- A range is written as start and end position: e.g., 3-5
  - Either can be omitted
  - The first character or field is numbered 1, not 0
- Example: select usernames of logged in users:

$ **who | cut -d" " -f1 | sort -u**

Expanding Tabs to Spaces with expand

- Used to replace tabs with spaces in files
- Tab size (maximum number of spaces for each tab) can be set with -t *number*
  - Default tab size is 8
- To only change tabs at the beginning of lines, use -i
- Example: change all tabs in *foo.txt* to three spaces, display it to the screen:

  $ **expand -t 3 foo.txt**

  $ **expand -3 foo.txt**

Using fmt to Format Text Files

- Arranges words nicely into lines of consistent length
- Use -u to convert to uniform spacing
  - One space between words, two between sentences
- Use -w *width* to set the maximum line width in characters

- Defaults to 75

- Example: change the line length of *notes.txt* to a maximum of 70 characters, and display it on the screen:

  $ **fmt -w 70 notes.txt | less**

Reading the Start of a File with head

- Prints the top of its input, and discards the rest
- Set the number of lines to print with -n *lines* or -*lines*
  - Defaults to ten lines
- View the headers of a HTML document called *homepage.html*:

  $ **head homepage.html**
- Print the first line of a text file (two alternatives):

  $ **head -n 1 notes.txt**

  $ **head -1 notes.txt**

Reading the End of a File with tail

- Similar to head, but prints lines at the end of a file
- The -f option watches the file forever
  - Continually updates the display as new entries are appended to the end of the file
  - Kill it with Ctrl+C
- The option -n is the same as in head (number of lines to print)
- Example: monitor HTTP requests on a webserver:

  $ **tail -f /var/log/httpd/access_log**

Numbering Lines of a File with nl or cat

- Display the input with line numbers against each line
- There are options to finely control the formatting
- By default, blank lines aren't numbered
  - The option -ba numbers every line
  - cat -n also numbers lines, including blank ones

Dividing Files into Chunks with split

- Splits files into equal-sized segments
- Syntax: split [*options*] [*input*] [*output-prefix*]
- Use -l *n* to split a file into *n*-line chunks
- Use -b *n* to split into chunks of *n* bytes each

- Output files are named using the specified output name with *aa*, *ab*, *ac*, etc., added to the end of the prefix
- Example: Split *essay.txt* into 30-line files, and save the output to files *short_aa*, *short_ab*, etc:

  $ **split -l 30 essay.txt short_**

Reversing Files with tac

- Similar to cat, but in reverse
- Prints the last line of the input first, the penultimate line second, and so on
- Example: show a list of logins and logouts, but with the most recent events at the end:

  $ **last | tac**

Modifying Files with sed

- sed uses a simple script to process each line of a file
- Specify the script file with -f *filename*
- Or give individual commands with -e *command*
- For example, if you have a script called *spelling.sed* which corrects your most common mistakes, you can feed a file through it:

  $ **sed -f spelling.sed < report.txt > corrected.txt**

Substituting with sed

- Use the s/*pattern*/*replacement*/ command to substitute text matching the *pattern* with the *replacement*
  - Add the /g modifier to replace every occurrence on each line, rather than just the first one
- For example, replace ‘thru’ with ‘through’:

  $ **sed -e ’s/thru/through/g’ input-file > output-file**
- sed has more complicated facilities which allow commands to be executed conditionally
  - Can be used as a very basic (but unpleasantly difficult!) programming language

Put Files Side-by-Side with paste

- paste takes lines from two or more files and puts them in columns of the output
- Use -d *char* to set the delimiter between fields in the output
  - The default is tab

- Giving -d more than one-character sets different delimiters between each pair of columns

- Example: assign passwords to users, separating them with a colon:

  $ **paste -d: usernames passwords > .htpasswd**

Lab Tasks

Task-1

I. Type in the example on the cut slide to display a list of users logged in. (Try just who on its own first to see what is happening.)

II. Arrange for the list of usernames in who's output to be sorted, and remove any duplicates.

III. Try the command last to display a record of login sessions, and then try reversing it with tac. Which is more useful? What if you pipe the output into less?

IV. Use sed to correct the misspelling 'environment' to 'environment'. Use it on a test file, containing a few lines of text, to check it. Does it work if the misspelling occurs more than once on the same line?

V. Use nl to number the lines in the output of the previous question.

Task-2

I. Try making an empty file and using tail -f to monitor it. Then add lines to it from a different terminal, using a command like this:

   a. $ **echo "testing" >>filename**

II. Once you have written some lines into your file, use tr to display it with all occurrences of the letters A–F changed to the numbers 0–5.

III. Try looking at the binary for the ls command (*/bin/ls*) with less. You can use the -f option to force it to display the file, even though it isn't text.

IV. Try viewing the same binary with od. Try it in its default mode, as well as with the options shown on the slide for outputting in hexadecimal.

Task-3

I. Use the split command to split the binary of the ls command into 1Kb chunks. You might want to create a directory especially for the split files, so that it can all be easily deleted later.

II. Put your split ls command back together again, and run it to make sure it still works. You will have to make sure you are running the new copy of it, for example ./my_ls, and make sure that the program is marked as ‘executable’ to run it, with the following command:

$ **chmod a+rx my_ls**

Lab No. 7: Parameter Passing in Linux

Objectives

Understand and practice different methods of parameter passing in Linux shell scripts.

Theoretical Description

Positional parameters

n Linux, a positional parameter is an argument passed to a script or a function in the order in which they appear. Positional parameters are referenced within the script or function by their position number. The first argument is $1, the second argument is $2, and so on. The parameter $0 refers to the name of the script itself.

Accessing Positional Parameters:

- $0: The name of the script.
- $1 to $9: The first to the ninth positional parameters.
- ${10} and beyond: For positional parameters beyond the ninth, you need to use curly braces, e.g., ${10} for the tenth parameter.

Special Positional Parameters:

- $#: The number of positional parameters.
- $*: All positional parameters as a single word.
- $@: All positional parameters as separate quoted strings.
- "$*": All positional parameters as a single word (with quotes around the whole string).
- "$@": All positional parameters as separate quoted strings (with quotes around each parameter individually).

Example

```
#!/bin/bash
# A simple script to demonstrate positional parameters
echo "Script name: $0"
echo "First parameter: $1"
echo "Second parameter: $2"
echo "All parameters: $@"
echo "Number of parameters: $#"
```

Environment Variables

In Linux, an environment parameter, also known as an environment variable, is a dynamic-named value that can affect the way running processes will behave on a computer. They are part of the environment in which a process runs and can be used to pass configuration information to applications.

Example

```
#!/bin/bash
# script name: envvar.sh
echo "Environment Variable USER: $USER"
echo "Environment Variable HOME: $HOME"
```

common environment variables in Linux:

PATH: Specifies the directories where executable files are located. When a command is run, the system looks for the executable in the directories listed in PATH.

```
echo $PATH
```

HOME: The current user's home directory.

```
echo $HOME
```

USER: The current logged-in user's name.

```
echo $USER
```

SHELL: The path to the current user's shell program.

```
echo $SHELL
```

Lab Tasks

Task-1

Write a shell script that get input from user during script exaction by using a read command

Task-2

Write a shell script to define and use functions with a shell script with parameter

Lab No. 8: Unix Streams, Pipes and Redirects

Objectives

This lab will give overview of the use of Unix Streams, Pipes and Redirects.

Theoretical Description

Standard Files

- Processes are connected to three standard files
  - Standard output
  - Standard error
  - Standard input
- Many programs open other files as well

Standard Input

- Programs can read data from their **standard input** file
- Abbreviated to **stdin**
- By default, this reads from the keyboard
- Characters typed into an interactive program (e.g., a text editor) go to stdin

Standard Output

- Programs can write data to their **standard output** file
- Abbreviated to **stdout**
- Used for a program's normal output
- By default this is printed on the terminal

Standard Error

- Programs can write data to their **standard error** output
- Standard error is similar to standard output, but used for error and warning messages
- Abbreviated to **stderr**
- Useful to separate program output from any program errors
- By default, this is written to your terminal
  - So, it gets 'mixed in' with the standard output

Pipes

- A **pipe** channels the output of one program to the input of another

- Allows programs to be chained together
- Programs in the chain run concurrently

- Use the vertical bar: |
- Sometimes known as the ‘pipe’ character

- Programs don’t need to do anything special to use pipes
- They read from stdin and write to stdout as normal

- For example, pipe the output of echo into the program rev (which reverses each line of its input):

  $ **echo Happy Birthday! | rev**

  !yadhtriB yppaH

Connecting Programs to Files

- **Redirection** connects a program to a named file
- The < symbol indicates the file to read input from:

$ **wc < thesis.txt**

- The file specified becomes the program’s standard input

- The > symbol indicates the file to write output to:

$ **who > users.txt**

- The program’s standard output goes into the file
- If the file already exists, it is overwritten

- Both can be used at the same time:

  $ **filter < input-file > output-file**

Appending to Files

- Use >> to append to a file:

  $ **date >> log.txt**

- Appends the standard output of the program to the end of an existing file
- If the file doesn’t already exist, it is created

Redirecting Multiple Files

- Open files have numbers, called **file descriptors**
- These can be used with redirection
- The three standard files always have the same numbers:

Name Descriptor

Standard input 0

Standard output 1

Standard error 2

Redirection with File Descriptors

- Redirection normally works with stdin and stdout
- Specify different files by putting the file descriptor number before the redirection symbol:
  - To redirect the standard error to a file:

$ **program 2> file**

  - To combine standard error with standard output:

$ **program > file 2>&1**

  - To save both output streams:

$ **program > stdout.txt 2> stderr.txt**

- The descriptors 3–9 can be connected to normal files, and are mainly used in shell scripts

Running Programs with xargs

- xargs reads pieces of text and runs another program with them as its arguments
  - Usually its input is a list of filenames to give to a file processing program
- Syntax: xargs *command* [*initial args*]
- Use -l *n* to use *n* items each time the command is run
  - The default is 1
- xargs is very often used with input piped from find
- Example: if there are too many files in a directory to delete in one go, use xargs to delete them ten at a time:

$ **find /tmp/rubbish/ | xargs -l10 rm -f**

tee

- The tee program makes a 'T-junction' in a pipeline
- It copies data from stdin to stdout, and also to a file
- Like > and | combined
- For example, to save details of everyone's logins, and save Bob's logins in a separate file:

$ **last | tee everyone.txt | grep bob > bob.txt**

tee grep last *bob.txt*

everyone.txt

PIPE PIPE REDIRECT

Lab Tasks

Task-1

I. Try the example on the ‘Pipes’ slide, using rev to reverse some text.

II. Try replacing the echo command with some other commands which produce output (e.g., whoami).

III. What happens when you replace rev with cat? You might like to try running cat with no arguments and entering some text.

Task-2

I. Run the command ls --color in a directory with a few files and directories. Some Linux distributions have ls set up to always use the --color option in normal circumstances, but in this case we will give it explicitly.

II. Try running the same command, but pipe the output into another program (e.g., cat or less). You should spot two differences in the output. ls detects whether its output is going straight to a terminal (to be viewed by a human directly) or into a pipe (to be read by another program).

Lab No. 9: Searching Regular Expression (GREP)

Objectives

This lab will give overview the use of Regular Expressions for searching test files.

Theoretical Description

Searching Files with grep

- grep prints lines from files which match a pattern
- For example, to find an entry in the password file */etc/passwd* relating to the user 'nancy':

$ **grep nancy /etc/passwd**

- n grep has a few useful options:
  - -i makes the matching case-insensitive
  - -r searches through files in specified directories, recursively
  - -l prints just the names of files which contain matching lines
  - -c prints the count of matches in each file
  - -n numbers the matching lines in the output
  - -v re*v*erses the test, printing lines which don't match

10.2 Pattern Matching

- Use grep to find patterns, as well as simple strings
- Patterns are expressed as **regular expressions**
- Certain punctuation characters have special meanings
- For example this might be a better way to search for Nancy's entry in the password file:

$ **grep '^nancy' /etc/passwd**

- The caret (^) anchors the pattern to the start of the line
- In the same way, $ acts as an **anchor** when it appears at the end of a string, making the pattern match only at the end of a line

Matching Repeated Patterns

- Some regexp special characters are also special to the shell, and so need to be protected with quotes or backslashes
- We can match a repeating pattern by adding a modifier:

$ **grep -i 'continued\.*'**

- Dot (.) on its own would match any character, so to match an actual dot we escape it with \
- The * modifier matches the preceding character zero or more times
- Similarly, the \+ modifier matches one or more times

Matching Alternative Patterns

- Multiple subpatterns can be provided as alternatives, separated with \|, for example:

$ **grep 'fish\|chips\|pies' food.txt**

- The previous command finds lines which match at least one of the words
- Use \(...\) to enforce precedence:

$ **grep -i '\(cream\|fish\|birthday\) cakes' delicacies.txt**

- Use square brackets to build a **character class**:

$ **grep '[Jj]oe [Bb]loggs' staff.txt**

- Any single character from the class matches; and ranges of characters can be expressed as

  'a-z'

Extended Regular Expression Syntax

- egrep runs grep in a different mode
  - Same as grep -E
- Special characters don't have to be marked with \
  - So \+ is written +, \(...\) is written (...), etc
  - In extended regexps, \+ is a literal +

sed

- sed reads input lines, runs editing-style commands on them, and writes them to stdout
- sed uses regular expressions as patterns in substitutions
  - sed regular expressions use the same syntax as grep
- For example, to used sed to put # at the start of each line:

$ **sed -e 's/^/#/' < input.txt > output.txt**

- sed has simple substitution and translation facilities, but can also be used like a programming language

Lab Tasks

Task-1

I. Use grep to find information about the HTTP protocol in the file */etc/services*.

II. Usually this file contains some comments, starting with the '#' symbol. Use grep with the -v option to ignore lines starting with '#' and look at the rest of the file in less.

III. Add another use of grep -v to your pipeline to remove blank lines (which match the pattern ^$).

IV. Use sed (also in the same pipeline) to remove the information after the '/' symbol on each line, leaving just the names of the protocols and their port numbers.

Lab No. 10: Programming Fundamental, if-else, Loops

Objectives

This lab will give overview the use of if-else, for, While, do while loop shell scripts

Theoretical Description

if statement

The if...fi statement is the fundamental control statement that allows Shell to make decisions and execute statements conditionally.

Syntax

```
if [ expression]
then
   Statement(s) to be executed if expression is true
fi
```

Script:

```
#!/bin/sh

a=10
b=20

if [ $a == $b]
then
   echo "a is equal to b"
fi

if [ $a != $b ]
then
   echo "a is not equal to b"
fi
```

The above script will generate the following result –

A is not equal to b

11.3 if-else statement

If else statements are useful decision-making statements which can be used to select an option from a given set of options.

Syntax

```
if [ expression]
```

```
then
   Statement(s) to be executed if expression is true
else
   Statement(s) to be executed if expression is not true
fi
```

Script:

The above example can also be written using the if...else statement as follows −

```
#!/bin/sh

a=10
b=20

if [ $a == $b ]
then
   echo "a is equal to b"
else
   echo "a is not equal to b"
fi
```

Upon execution, you will receive the following result –

```
a is not equal to b
```

if – else-if

The if...elif...fi statement is the one level advance form of control statement that allows Shell to make correct decision out of several conditions.

Syntax

```
if [ expression 1]
then
   Statement(s) to be executed if expression 1 is true
elif [ expression 2]
then
   Statement(s) to be executed if expression 2 is true
elif [ expression 3]
then
   Statement(s) to be executed if expression 3 is true
else
   Statement(s) to be executed if no expression is true
Fi
```

Script:

```
#!/bin/sh

a=10
b=20

if [ $a == $b ]
then
   echo "a is equal to b"
elif [ $a -gt $b ]
then
   echo "a is greater than b"
elif [ $a -lt $b ]
then
   echo "a is less than b"
else
   echo "None of the condition met"
fi
```

Upon execution, you will receive the following result –

```
a is less than b
```

Loops

In Unix shell scripting, loops are used to execute a block of code repeatedly. The most commonly used loops are the for, while, and until loops. Here are detailed examples for each type of loop.

for Loop

A for loop iterates over a list of values and executes a block of code for each value.

Syntax

```
for var in word1 word2 ... wordN
do
   Statement(s) to be executed for every word.
done
```

Script:

```
#!/bin/sh

for var in 0 1 2 3 4 5 6 7 8 9
do
   echo $var
done
```

while Loop

The while loop enables you to execute a set of commands repeatedly until some condition occurs. It is usually used when you need to manipulate the value of a variable repeatedly.

Syntax

```
while command
do
   Statement(s) to be executed if command is true
Done
```

Script

```
#!/bin/sh

a=0

while [ $a -lt 10 ]
do
   echo $a
   a=`expr $a + 1`
done
```

Lab Tasks

Task-1

I. Write a unix shell script to print sum of numbers from 1 to 10 by using while Loop.

II. Write a unix shell script to print Product of even numbers from 1 to 15 by using for Loop.

III. Write a unix shell script to print Multiplication table of a numbers that is read by user by using Loop.

IV. Write a unix shell script that read a string from user and print in reverse order.

Lab No. 11: Switch case statement, Functions

Objectives

This lab will give overview the use of Switch case structure, functions and Various Programming related exercises in Linux.

Theoretical Description

switch case structure

The bash case statement is the simplest form of the if elif else conditional statement. The case statement simplifies complex conditions with multiple different choices. This statement is easier to maintain and more readable than nested if statements.

Syntax

```
case $variable in
 pattern-1) commands;;
 pattern-2) commands;;
pattern-3) commands;;
pattern-N) commands;;
 *) commands;;
Esac
```

The case statement starts with the case keyword followed by the $variable and the in keyword. The statement ends with the case keyword backwards - esac.

$variable

- The script compares the input $variable against the patterns in each clause until it finds a match.

Patterns

- A pattern and its commands make a **clause**, which ends with ;;.
- Patterns support special characters.
- The) operator terminates a pattern list.
- The | operator separates multiple patterns.
- The script executes the commands corresponding to the first pattern matching the input $variable.
- The asterisk * symbol defines the default case, usually in the final pattern.

Exit Status

The script has two exit statuses:

- **0**. The return status when the input matches no pattern.
- **Executed command status**. If the command matches the input variable to a pattern, the executed command exit status is returned.

Program:

```
#!/bin/bash

echo "Which color do you like best?"

 echo "1 - Blue"

echo "2 - Red"

echo "3 - Yellow"

echo "4 - Green"

echo "5 - Orange"

read color;

case $color in 1)

 echo "Blue is a primary color.";;

2)

 echo "Red is a primary color.";;

3)

 echo "Yellow is a primary color.";;

4)

echo "Green is a secondary color.";;

 5)

echo "Orange is a secondary color.";;

 *)
```

```
echo "This color is not available. Please choose a different one.";;
esac
```

Functions

A function is a collection of statements that execute a specified task. Its main goal is to break down a complicated procedure into simpler subroutines that can subsequently be used to accomplish the more complex routine. For the following reasons, functions are popular:

- Assist with code reuse.
- Enhance the program's readability.
- Modularize the software.
- Allow for easy maintenance.

Syntax

```
function_name(){
    // body of the function
}
```

The function_name can be any valid string and the body can be any sequence of valid statements in the scripting language.

Program:

```
echo -n "Enter Left-End: "
read le
echo -n "Enter Right-End: "
read ri
is_prime(){
  if [ $1 -lt 2 ]; then
     return
  fi
```

```
  ctr=0
  for((i=2;i<$1;i++)){
     if [ $(( $1 % i )) -eq 0 ]; then
        ctr=$(( ctr +1 ))
     fi
  }
  if [ $ctr -eq 0 ]; then


printf "%d " "$1"
  fi
}
printf "Prime Numbers between %d and %d are: " "$le" "$ri"
for((i=le;i<=ri;i++)){
  is_prime $i
}
printf "\n"
```

Types of Function

The functions in shell scripting can be boxed into a number of categories. The following are some of them:

1. The functions that return a value to the caller. The return keyword is used by the functions for this purpose.

```
find_avg(){
  len=$#
  sum=0
  for x in "$@"
```

```
  do
    sum=$((sum + x))
  done
  avg=$((sum/len))
  return $avg
}
find_avg 30 40 50 60
printf "%f" "$?"
printf "\n"
```

2. The functions that terminate the shell using the exit keyword.

```
is_odd(){
  x=$1
  if [ $((x%2)) == 0 ]; then
    echo "Invalid Input"
    exit 1
  else
    echo "Number is Odd"
  fi
}
is_odd 64
```

3. The functions that alter the value of a variable or variables.

```
a=1
increment(){
  a=$((a+1))
```

```
  return
}
increment
echo "$a"
```

4. The functions that echo output to the standard output.

```
hello_world(){
  echo "Hello World"
  return
}
hello_world
```

Lab Tasks

Task-1

Write a shell script that prompts the user to enter a day of the week and then prints a corresponding message for that day. The script will utilize a case statement to handle the different possible inputs.

Task-2

Write a shell script that define the following functions

- **greet:** A function that takes a name as an argument and prints a greeting message.
- **sum:** A function that takes two numbers as arguments and prints their sum.
- **is_even:** A function that takes a number as an argument and prints whether it is even or odd.

Lab No. 12: File Management using System calls

Objectives

This lab will give overview the use of System calls, Implementation of System calls using GCC Compiler in Linux.

Theoretical Description

System Calls

System calls are the calls that a program makes to the system kernel to provide the services to which the program does not have direct access. **Example**
providing access to input and output devices such as monitors and keyboards. We can use various functions provided in the C Programming language for input/output system calls such as create, open, read, write, etc.

Important Terminology

Some of the common Terms regarding File

File Descriptor

The file descriptor is an integer that uniquely identifies an open file of the process.

File Descriptor Table

A file descriptor table is the collection of integer array indices that are file descriptors in which elements are pointers to file table entries. One unique file descriptors table is provided in the operating system for each process.

File Table entry

File table entries are a structure In-memory surrogate for an open file, which is created when processing a request to open the file and these entries maintain file position.

Input/Output System Calls

There is total 5 I/O System calls

- **C create()**

The create() function is used to create a new empty file in C. We can specify the permission and the name of the file which we want to create using the create() function. It is defined inside **<unistd.h>** header file and the flags that are passed as arguments are defined inside **<fcntl.h>** header file.

Syntax

int create(char *filename, mode_t mode);

Parameter

- **filename:** name of the file which you want to create
- **mode:** indicates permissions of the new file.

Return value

- return first unused file descriptor (generally 3 when first creating use in the process because 0, 1, 2 fd are reserved)
- return -1 when an error

- **open()**

  The open() function in C is used to open the file for reading, writing, or both. It is also capable of creating the file if it does not exist. It is defined inside **<unistd.h>** header file and the flags that are passed as arguments are defined inside **<fcntl.h>** header file.

  Syntax

  int **open** (const char* *Path*, int *flags*);

  Parameter

  - **Path:** Path to the file which we want to open.
    - Use the **absolute path** beginning with "/" when you are **not working in the same directory** as the C source file.
    - Use **relative path** which is only the file name with extension, when you are **working in the same directory** as the C source file.
  - **flags:** It is used to specify how you want to open the file. We can use the following flags.

Program:

```
// C program to illustrate
// open system call
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
```

```
extern int errno;
int main()
{
   // if file does not have in directory
   // then file foo.txt is created.
   int fd = open("foo.txt", O_RDONLY | O_CREAT);
    printf("fd = %d\n", fd);
    if (fd == -1) {
       // print which type of error have in a code
       printf("Error Number % d\n", errno);

       // print program detail "Success or failure"
       perror("Program");
   }
   return 0;
}
```

- **Close()**

The close() function in C tells the operating system that you are done with a file descriptor and closes the file pointed by the file descriptor. It is defined inside **<unistd.h>** header file.

Syntax

```
int close(int fd);
```

Parameter

- **fd: F**ile descriptor of the file that you want to close.

Program:

```
// C program to illustrate close system Call
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
   int fd1 = open("foo.txt", O_RDONLY);
```

```
    if (fd1 < 0) {
        perror("c1");
        exit(1);
    }
    printf("opened the fd = % d\n", fd1);

    // Using close system Call
    if (close(fd1) < 0) {
        perror("c1");
        exit(1);
    }
    printf("closed the fd.\n");
}
```

- **Read()**

From the file indicated by the file descriptor fd, the read() function reads the specified amount of bytes **cnt** of input into the memory area indicated by **buf**. A successful read() updates the access time for the file. The read() function is also defined inside the <unistd.h> header file.

Syntax

size_t **read** (int *fd*, void* *buf*, size_t *cnt*);

Parameter

- **fd:** file descriptor of the file from which data is to be read.
- **buf:** buffer to read data from
- **cnt:** length of the buffer

Program

```
// C program to illustrate
// read system Call
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
```

```
    int fd, sz;
    char* c = (char*)calloc(100, sizeof(char));

    fd = open("foo.txt", O_RDONLY);
    if (fd < 0) {
        perror("r1");
        exit(1);
    }
     sz = read(fd, c, 10);
    printf("called read(% d, c, 10). returned that"
           " %d bytes were read.\n",
           fd, sz);
    c[sz] = '\0';
    printf("Those bytes are as follows: % s\n", c);

    return 0;
}
```

- **Write()**

Writes cnt bytes from buf to the file or socket associated with fd. cnt should not be greater than INT_MAX (defined in the limits.h header file). If cnt is zero, write() simply returns 0 without attempting any other action.

The write() is also defined inside **<unistd.h>** header file.

Syntax

size_t **write** (int *fd*, void* *buf*, size_t *cnt*);

Parameter

- **fd:** file descriptor
- **buf:** buffer to write data from.
- **cnt:** length of the buffer.

Program:

```
// C program to illustrate
// I/O system Calls
```

```
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
 int main(void)
{
   int fd[2];
   char buf1[12] = "hello world";
   char buf2[12];
    // assume foobar.txt is already created
   fd[0] = open("foobar.txt", O_RDWR);
   fd[1] = open("foobar.txt", O_RDWR);

   write(fd[0], buf1, strlen(buf1));
   write(1, buf2, read(fd[1], buf2, 12));

   close(fd[0]);
   close(fd[1]);

   return 0;
}
```

Lab Tasks

Task-1

Write a program that read contents of a file and display on screen.

Task-2

write a shell script that read data from one file and write into second file

Lab No. 13: System Calls in Linux

Objectives

This lab will give overview the use of System calls, fork (), getpid(), getppid(), wait(), opendir(), readdir(), closedir()

Theoretical Description

Fork()

fork() is used to create a new process. The newly created process is called the child process, and the process that called fork() is the parent process.

Some of the important points on fork() are as follows.

- The parent will get the child process ID with non-zero value.
- Zero Value is returned to the child.
- If there will be any system or hardware errors while creating the child, -1 is returned to the fork().
- With the unique process ID obtained by the child process, it does not match the ID of any existing process group.

Example

```
pid_t pid;
pid = fork();
if (pid < 0) {
   // Error occurred
   fprintf(stderr, "Fork failed\n");
   return 1;
} else if (pid == 0) {

   // Child process
   printf("This is the child process.\n");
} else {
```

```
    // Parent process
    printf("This is the parent process.\n");
}
```

Getpid()

getpid() returns the process ID (PID) of the calling process.

Example

```
pid_t pid;
pid = getpid();
printf("The process ID is %d\n", pid);
```

getppid()

getppid() returns the process ID (PID) of the parent of the calling process.

Example

```
pid_t ppid;
ppid = getppid();
printf("The parent process ID is %d\n", ppid);
```

wait()

wait() makes the calling process wait until one of its child processes terminates. It returns the process ID of the terminated child process.

Example

```
pid_t pid, wpid;
int status;
pid = fork();
if (pid == 0) {
    // Child process
    printf("Child process is running.\n");
    sleep(2); // Simulate some work
    printf("Child process is terminating.\n");
    _exit(0);
} else if (pid > 0) {
    // Parent process
    wpid = wait(&status);
    printf("Child process with PID %d has terminated.\n", wpid);
```

```
}
```

Opendir()

opendir() opens a directory stream corresponding to the directory name, and returns a pointer to the directory stream.

Example

```
DIR *dir;
dir = opendir("/path/to/directory");

if (dir == NULL) {
    perror("opendir");
    return 1;
}
```

Readdir()

readdir() reads the next directory entry from the directory stream opened by opendir().

Example

```
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    printf("Found file: %s\n", entry->d_name);
}
```

Closedir()

closedir() closes the directory stream opened by opendir().

```
if (closedir(dir) == -1) {
    perror("closedir");
    return 1;
}
```

Lab Tasks

Task-1

Write a C program that demonstrates the creation of a child process using the fork() system call.

Task-2

Write a C program that open a directory, read next entry from directory and then close the directory.